Experiment 5:

Title: To develop a React application that demonstrates form handling using multiple input types and state management.

1. Create the New Project

Open your terminal or command prompt and run these commands to scaffold a new project using Vite

```
# Create the project folder
npm create vite@latest student-registration -- --template react

# Move into the project directory
cd student-registration

# Install necessary dependencies
npm install

# Open in VS Code (optional)
code .

# Start the development server
npm run dev
```

---

2. Create the Component

Create a new file in `src/StudentForm.jsx` and paste the logic. This version includes a Live Preview section so you can see the state management working in real-time.

JavaScript

```javascript
import React, { useState } from "react";

export default function StudentForm() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
    age: "",
    dob: "",
    gender: "",
    course: "",
    skills: [],
    bio: "",
    file: null
  });

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    if (type === "checkbox") {
      let updatedSkills = [...formData.skills];
      if (checked) {
        updatedSkills.push(value);
      } else {
        updatedSkills = updatedSkills.filter(skill => skill !== value);
      }
      setFormData({ ...formData, skills: updatedSkills });
    } else {
      setFormData({ ...formData, [name]: value });
    }
  };

  const handleFileChange = (e) => {
    setFormData({ ...formData, file: e.target.files[0] });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Final Form Data:", formData);
    alert("Form Submitted! Check Console.");
```

```
  };

  return (
    <div style={{ display: "flex", gap: "40px", padding: "20px", fontFamily: "Arial" }}>
      {/* Form Section */}
      <div style={{ flex: 1, border: "1px solid #ddd", padding: "20px", borderRadius: "8px" }}>
        <h2>Student Registration</h2>
        <form onSubmit={handleSubmit}>
          <label>Name: </label> <input type="text" name="name" onChange={handleChange}
/><br/><br/>
          <label>Email: </label> <input type="email" name="email" onChange={handleChange}
/><br/><br/>
          <label>Gender: </label>
          <input type="radio" name="gender" value="Male" onChange={handleChange}/> Male
          <input type="radio" name="gender" value="Female" onChange={handleChange}/>
Female<br/><br/>
          <label>Course: </label>
          <select name="course" onChange={handleChange}>
            <option value="">Select</option>
            <option value="BTech">BTech</option>
            <option value="MCA">MCA</option>
          </select><br/><br/>
          <label>Skills: </label>
          <input type="checkbox" value="JavaScript" onChange={handleChange}/> JS
          <input type="checkbox" value="React" onChange={handleChange}/> React<br/><br/>
          <label>Bio: </label><br/>
          <textarea name="bio" onChange={handleChange}></textarea><br/><br/>
          <button type="submit">Submit</button>
        </form>
      </div>

      {/* Live Preview Section */}
      <div style={{ flex: 1, background: "#f9f9f9", padding: "20px", borderRadius: "8px" }}>
        <h2>Live Preview (State)</h2>
        <p><strong>Name:</strong> {formData.name}</p>
        <p><strong>Email:</strong> {formData.email}</p>
        <p><strong>Course:</strong> {formData.course}</p>
        <p><strong>Skills:</strong> {formData.skills.join(", ")}</p>
      </div>
    </div>
  );
}
```

3. Update App.jsx

Clean out the default boilerplate in `src/App.jsx` and replace it with this:

JavaScript

```
import StudentForm from "./StudentForm";

function App() {
  return (
    <div className="App">
      <h1 style={{ textAlign: "center" }}>Experiment 5: Form Handling</h1>
      <StudentForm />
    </div>
  );
}

export default App;
```

---

4. Execution

1. **Ensure your terminal says the server is running (usually at `http://localhost:5173`).**
2. **Type into the input fields.**
3. **Observe the Live Preview section—it updates instantly because of the `useState` hook.**
4. **Open your browser console (F12) and click Submit to see the structured object.**